

DL & GAI Lab — Quick Reference Writeups (Programs 1–10)

How to use in exam: the paper won't say "Program 3" — it'll describe a *dataset + task*. Match it below, pick that write-up.

If the question mentions...	Use Program
Small binary dataset, linear separability, AND/OR/XOR, "single neuron", sklearn <code>Perceptron()</code>	1
Iris / Ionosphere, backpropagation, "compare activation functions"	2
MNIST/CIFAR-10/Fashion-MNIST, plain Dense layers, "Deep Feed Forward", accuracy @ 5/50/100 epochs	3
Fashion-MNIST/CIFAR, "CNN with 4+ conv layers", image classification	4
CIFAR-10, "regularization", "dropout", "overfitting"	5
MNIST, "LeNet/AlexNet/VGG", "Transfer Learning", pretrained weights	6
IMDB reviews, "sentiment analysis", RNN/LSTM/GRU/Bi-LSTM	7
"Generate images", GAN, generator/discriminator, MNIST/Fashion-MNIST	8
"Fine-tune GPT-2", Shakespeare/custom text, text generation	9
WikiText / prompt engineering, LLM fine-tuning, custom prompts	10

Program 1 — Single/Multi-Unit Perceptron (Linear Classifiers)

Aim: Classify a linearly separable binary dataset using a single-unit perceptron (no custom model — sklearn `Perceptron`); show it fails on XOR (non-linear) and fix with a 2-layer MLP.

Theory: A perceptron is the simplest neural unit: $\text{output} = \text{step}(\sum w_i x_i + b)$. It draws **one straight decision boundary**, so it can only separate **linearly separable** classes (AND, OR). XOR's positive and negative points cannot be split by a single line, so a single perceptron's accuracy caps at ~50-75%. Adding a **hidden layer** (Multi-Layer Perceptron) lets the network combine two lines into a non-linear boundary, solving XOR — this is why MLPs + backpropagation replaced single perceptrons.

Dataset loading:

```
import numpy as np
from sklearn.linear_model import Perceptron
X = np.array([[0,0],[0,1],[1,0],[1,1]]) # replace with given CSV via
pd.read_csv if provided
y = np.array([0,1,1,1]) # OR labels
```

Model + training (input/output layer = perceptron weights):

```
p = Perceptron()
p.fit(X, y)
print("Weights:", p.coef_, "Bias:", p.intercept_)
print("Accuracy:", p.score(X, y))
```

XOR fix — 2-layer NN with sigmoid activation, manual backprop (key activation/training code):

```
def sigmoid(x): return 1/(1+np.exp(-x))
def sigmoid_deriv(x): return x*(1-x)

W1 = np.random.randn(2,2); b1 = np.zeros((1,2))
W2 = np.random.randn(2,1); b2 = np.zeros((1,1))
y_xor = np.array([[0],[1],[1],[0]])

for epoch in range(10000):
    hidden = sigmoid(X @ W1 + b1)
    output = sigmoid(hidden @ W2 + b2)
    error = y_xor - output
    d_out = error * sigmoid_deriv(output)
    d_hid = (d_out @ W2.T) * sigmoid_deriv(hidden)
    W2 += hidden.T @ d_out; b2 += d_out.sum(0, keepdims=True)
    W1 += X.T @ d_hid; b1 += d_hid.sum(0, keepdims=True)
```

Output:

```
OR: Weights: [[2. 2.]] Bias: [-1.] Accuracy: 1.0
AND: Weights: [[2. 2.]] Bias: [-2.] Accuracy: 1.0
XOR (single perceptron): Weights: [[0. 0.]] Bias: [0.] Accuracy: 0.5
→ FAILS (not linearly separable)
XOR (2-layer MLP): Predicted [[0.][1.][1.][0.]] → CORRECT
```

Conclusion: single perceptron only solves linearly separable problems (AND/OR); XOR needs a hidden layer.

Program 2 — ANN with Backpropagation (Activation Function Comparison)

Aim: Build an ANN using Keras Backpropagation, vary activation functions, compare accuracy on Iris dataset.

Theory: Backpropagation trains a multi-layer ANN by computing the loss at the output, then propagating the gradient backward layer-by-layer (chain rule) to update every weight via gradient descent. The **activation function** injects non-linearity between layers: **Sigmoid** (0-1 range) suffers from vanishing gradients in deeper nets; **Tanh** (-1 to 1) is zero-centered but still saturates; **ReLU** ($\max(0, x)$) avoids saturation for positive inputs and trains fastest — hence it usually wins in this comparison.

Dataset loading:

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.utils import to_categorical

X, y = datasets.load_iris(return_X_y=True)
y = to_categorical(y)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
X_train = StandardScaler().fit_transform(X_train)
X_test = StandardScaler().fit(X_train).transform(X_test)
```

Model (input/hidden/output layers + activation):

```
from keras.models import Sequential
from keras.layers import Dense

def create_model(activation):
    model = Sequential()
    model.add(Dense(16, input_dim=4, activation=activation))    #
    hidden layer 1
    model.add(Dense(8, activation=activation))                  #
    hidden layer 2
    model.add(Dense(3, activation='softmax'))                   #
    output layer
    model.compile(loss='categorical_crossentropy', optimizer='sgd',
metrics=['accuracy'])
    return model
```

Training + optimizer/config comparison:

```
for act in ['sigmoid', 'relu', 'tanh']:
    model = create_model(act)
    model.fit(X_train, y_train, epochs=100, verbose=0)
    loss, acc = model.evaluate(X_test, y_test, verbose=0)
    print(act, "-> Accuracy:", round(acc, 4))
```

Output:

```
sigmoid -> Accuracy: 0.30 (slow convergence)
relu     -> Accuracy: 0.93
tanh     -> Accuracy: 0.90
```

Conclusion: ReLU/tanh converge faster than sigmoid for this shallow network; optimizer (SGD vs Adam) and learning rate also affect convergence speed.

Program 3 — Deep Feed-Forward NN for Image Classification

Aim: Classify MNIST / Fashion-MNIST / CIFAR-10 using a Deep FFNN (Dense layers only); record accuracy at epochs 5, 50, 100.

Theory: A Deep Feed-Forward Network (DFFNN) stacks several fully-connected (Dense) layers, each layer's neurons connected to every neuron in the previous layer, with **no cycles/memory** (feed-forward only). Images must be **flattened** to a 1-D vector before being fed in, so spatial structure (pixel neighborhoods) is lost — this is why DFFNNs work reasonably on simple datasets (MNIST) but plateau on complex, spatially-rich ones (CIFAR-10), motivating CNNs.

Dataset loading + preprocessing:

```
from tensorflow import keras
(X_train, y_train), (X_test, y_test) =
keras.datasets.mnist.load_data()
X_train = X_train.reshape(-1, 784).astype("float32") / 255
X_test  = X_test.reshape(-1, 784).astype("float32") / 255
```

Model (input → 3 hidden Dense+ReLU → output softmax):

```
from keras.models import Sequential
from keras.layers import Dense, Input

def build_model():
    model = Sequential()
    model.add(Input(shape=(784,)))
    model.add(Dense(128, activation='relu',
kernel_initializer='he_normal'))
    model.add(Dense(64, activation='relu',
kernel_initializer='he_normal'))
    model.add(Dense(32, activation='relu',
kernel_initializer='he_normal'))
    model.add(Dense(10, activation='softmax'))
    model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy', metrics=['accuracy'])
    return model
```

Training across epoch settings:

```
for ep in [5, 50, 100]:
    model = build_model()
    model.fit(X_train, y_train, epochs=ep, batch_size=32,
validation_split=0.2)
    print(ep, model.evaluate(X_test, y_test, verbose=0))
```

(For CIFAR-10, flatten to 3072 or better use Conv2D — see Program 4.)

Output:

```
MNIST:   5 ep: Train 0.98 / Test 0.968   |   50 ep: 0.994 / 0.977   |  
100 ep: 0.996 / 0.981  
Fashion: 5 ep: Train 0.876 / Test 0.853 | ...  
CIFAR-10 (dense): ~0.65 test accuracy (plateaus – images need CNN, not  
plain Dense)
```

Conclusion: accuracy rises then plateaus/overfits with more epochs; CNNs outperform Dense-only models on image data (esp. CIFAR-10).

Program 4 — CNN with 4+ Convolution Layers (Image Classification)

Aim: Build a CNN with 4+ Conv layers for Fashion-MNIST; compare epochs 5, 10, 20.

Theory: A CNN preserves spatial structure using **Conv2D** layers (small learnable filters that slide over the image detecting edges/textures/shapes), followed by **MaxPooling** (downsamples, keeps the strongest activation, adds translation invariance). Stacking multiple conv layers lets earlier layers learn low-level features (edges) and deeper layers learn high-level features (shapes/parts). **Flatten + Dense** layers at the end map these learned feature maps to class scores.

Dataset loading:

```
from keras.datasets import fashion_mnist  
from keras.utils import to_categorical  
(train_X, train_Y), (test_X, test_Y) = fashion_mnist.load_data()  
train_X = train_X.reshape(-1,28,28,1).astype('float32')/255  
test_X = test_X.reshape(-1,28,28,1).astype('float32')/255  
train_Y_oh, test_Y_oh = to_categorical(train_Y),  
to_categorical(test_Y)
```

Model — 4 Conv2D blocks + Dense output (activation=relu/softmax):

```
from keras.models import Sequential  
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense,  
Activation  
  
def create_model():  
    model = Sequential()  
    model.add(Conv2D(256, (3,3), padding='same',  
input_shape=(28,28,1))); model.add(Activation('relu'));  
    model.add(MaxPooling2D(2,2))  
    model.add(Conv2D(128, (3,3), padding='same'));  
    model.add(Activation('relu')); model.add(MaxPooling2D(2,2))  
    model.add(Conv2D(64, (3,3), padding='same'));  
    model.add(Activation('relu'))  
    model.add(Conv2D(32, (3,3), padding='same'));
```

```

model.add(Activation('relu'))
model.add(Flatten())
model.add(Dense(64, activation='relu'))
model.add(Dense(10, activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
return model

```

Training:

```

for ep in [5, 10, 20]:
    model = create_model()
    model.fit(train_X, train_Y_oh, batch_size=64, epochs=ep)
    print(ep, model.evaluate(test_X, test_Y_oh, verbose=0))

```

Output:

5 epochs: accuracy $\approx$ 0.92 (train), loss dropping 0.45 $\rightarrow$ 0.20
 10 epochs: accuracy $\approx$ 0.93–0.94
 20 epochs: accuracy $\approx$ 0.94–0.95 (diminishing returns, GPU strongly recommended – CPU $\sim$ 800s/epoch vs GPU $\sim$ 10–15s/epoch)

Program 5 — CNN with Regularization (L1/L2) and Dropout

Aim: Build a CNN for CIFAR-10; add L1, L2 regularization and Dropout separately, then combined; compare train/test accuracy to study overfitting control.

Theory: Overfitting happens when a model memorizes training data instead of generalizing (high train accuracy, low test accuracy). **L1 regularization** adds $\lambda \sum |w|$ to the loss, pushing many weights to exactly zero (sparse weights, feature selection). **L2 regularization** adds $\lambda \sum w^2$, shrinking weights smoothly without zeroing them (weight decay). **Dropout** randomly disables a fraction of neurons each training step, forcing the network to not rely on any single neuron, which improves generalization. Too strong a penalty (e.g., L1=0.01 on many layers) can underfit instead.

Dataset:

```

from keras.datasets import cifar10
from keras.utils import to_categorical
(train_X, train_Y), (test_X, test_Y) = cifar10.load_data()
train_X, test_X = train_X.astype('float32')/255,
test_X.astype('float32')/255
train_Y_oh, test_Y_oh = to_categorical(train_Y),
to_categorical(test_Y)

```

Base CNN (no regularization):

```

from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense,
Activation, Dropout

```

```

from keras.regularizers import l1, l2

model = Sequential()
model.add(Conv2D(256,(3,3), input_shape=(32,32,3)));
model.add(Activation('relu')); model.add(MaxPooling2D((2,2)))
model.add(Conv2D(128,(3,3))); model.add(Activation('relu'));
model.add(MaxPooling2D((2,2)))
model.add(Conv2D(64,(3,3))); model.add(Activation('relu'))
model.add(Conv2D(28,(3,3))); model.add(Activation('relu'))
model.add(Flatten())
model.add(Dense(64)); model.add(Activation('relu'))
model.add(Dense(10)); model.add(Activation('softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])

```

Add L2 + Dropout (best-performing combo, swap in wherever a Conv/Dense layer is defined):

```

Conv2D(256,(3,3), input_shape=(32,32,3), kernel_regularizer=l2(0.01))
Dropout(0.20)    # after each pooling/dense layer

```

For L1: kernel_regularizer=l1(0.01) (same placement).

Training: model.fit(train_X, train_Y_oh, batch_size=64, epochs=5)

Output (5 epochs each):

```

Base model:      Train acc ≈ 0.70, Test acc ≈ 0.66
L1 regularizer:  Accuracy stuck ≈ 0.10 (0.01 penalty too high →
underfitting)
L2 regularizer:  Accuracy ≈ 0.42–0.45 (better than L1, still
constrained)
Dropout only:    Accuracy ≈ 0.59–0.65 (best generalization, mild
train/test gap)
L2 + Dropout:    Accuracy ≈ 0.33–0.40 (double regularization slows
convergence at 5 epochs)

```

Conclusion: Dropout alone gave the best trade-off here; L1=0.01 was too strong and under-fit; regularization strength must be tuned to epoch budget.

Program 6 — Transfer Learning (LeNet, AlexNet, VGG)

Aim: Implement LeNet from scratch and use pretrained AlexNet/VGG16 (transfer learning, frozen conv base) to classify MNIST; compare accuracy across epochs 5/50/(250).

Theory: LeNet (1998) is the earliest CNN — 2 conv+pool blocks + 3 FC layers, designed for small digit images. **AlexNet** (2012) and **VGG** (2014) are much deeper CNNs pretrained on ImageNet (millions of images). **Transfer learning** reuses these pretrained convolutional filters (which already know generic edge/texture/shape features) by **freezing** them

(requires_grad=False) and only training a new final classifier layer for the target task — this needs far less data/time than training a deep net from scratch, and usually converges faster/higher.

Dataset + transforms:

```
import torch, torch.nn as nn, torch.optim as optim
from torchvision import datasets, transforms, models
from torch.utils.data import DataLoader

tf_lenet = transforms.Compose([transforms.Resize((32,32)),
transforms.ToTensor(), transforms.Normalize((0.5,), (0.5,))])
tf_transfer = transforms.Compose([transforms.Resize((224,224)),
transforms.Grayscale(3), transforms.ToTensor(),
transforms.Normalize((0.5,), (0.5,))])

train_ds = datasets.MNIST('./data', train=True, download=True,
transform=tf_lenet)
train_loader = DataLoader(train_ds, batch_size=128, shuffle=True)
```

LeNet (built from scratch):

```
class LeNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
            nn.Conv2d(1,6,5), nn.Tanh(), nn.AvgPool2d(2),
            nn.Conv2d(6,16,5), nn.Tanh(), nn.AvgPool2d(2),
            nn.Flatten(),
            nn.Linear(16*5*5,120), nn.Tanh(),
            nn.Linear(120,84), nn.Tanh(),
            nn.Linear(84,10))
    def forward(self, x): return self.model(x)
```

Transfer learning — freeze conv base, replace classifier head:

```
alexnet = models.alexnet(weights=models.AlexNet_Weights.DEFAULT)
for p in alexnet.features.parameters(): p.requires_grad = False #
freeze
alexnet.classifier[6] = nn.Linear(4096, 10) #
new output layer

vgg16 = models.vgg16(weights=models.VGG16_Weights.DEFAULT)
for p in vgg16.features.parameters(): p.requires_grad = False
vgg16.classifier[6] = nn.Linear(4096, 10)
```

Training config/optimizer (only unfrozen params updated):

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(filter(lambda p: p.requires_grad,
model.parameters()), lr=0.001)
```



```

for epoch in range(epochs):
    for images, labels in train_loader:
        optimizer.zero_grad()
        loss = criterion(model(images), labels)
        loss.backward(); optimizer.step()

```

Output:

LeNet: Epoch 1: 96.4% Epoch 5: 98.4% Epoch 50: ~99.0%
 AlexNet (transfer, 10 ep): ~99.0%
 VGG16 (transfer, 10 ep): ~99.2%

Conclusion: pretrained AlexNet/VGG16 (transfer learning) converge to high accuracy faster than LeNet trained from scratch, since conv filters are already learned.

Program 7 — Sentiment Analysis using RNN / LSTM / Bi-LSTM / GRU

Aim: Classify IMDB movie review sentiment (binary) using Embedding + RNN variants; compare accuracy and ROC-AUC.

Theory: Text is sequential, so an **Embedding layer** first converts word indices into dense vectors capturing semantic meaning. A **SimpleRNN** processes the sequence step-by-step, carrying a hidden state forward, but suffers from **vanishing gradients** on long sequences (early words get “forgotten”). **LSTM** fixes this with gated memory cells (input/forget/output gates) that control what to remember/discard. **GRU** is a simplified LSTM (2 gates instead of 3, faster, similar accuracy). **Bidirectional** wraps a recurrent layer to read the sequence both forward and backward, capturing context from both directions.

Dataset loading:

```

from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences

vocab_size, maxlen = 10000, 200
(X_train, y_train), (X_test, y_test) =
imdb.load_data(num_words=vocab_size)
X_train = pad_sequences(X_train, maxlen=maxlen)
X_test  = pad_sequences(X_test,  maxlen=maxlen)

```

Models (Embedding input layer → recurrent layer → sigmoid output):

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, LSTM, GRU,
Bidirectional, Dense

rnn_model = Sequential([Embedding(vocab_size,128), SimpleRNN(64),
Dense(1, activation='sigmoid')])
lstm_model = Sequential([Embedding(vocab_size,128), LSTM(64),
Dense(1, activation='sigmoid')])

```

```

bilstm_model = Sequential([Embedding(vocab_size,128),
Bidirectional(LSTM(64)), Dense(1, activation='sigmoid')])
gru_model    = Sequential([Embedding(vocab_size,128), GRU(64),
Dense(1, activation='sigmoid')])

```

Compile + train (loss/optimizer config):

```

def compile_and_train(model, name):
    model.compile(loss='binary_crossentropy', optimizer='adam',
metrics=['accuracy'])
    model.fit(X_train, y_train, epochs=3, batch_size=128,
validation_data=(X_test, y_test))
    return model.evaluate(X_test, y_test)

```

Output:

```

RNN      : Test Accuracy ≈ 0.8518
LSTM     : Test Accuracy ≈ 0.8671
Bi-LSTM  : Test Accuracy ≈ 0.8659
GRU      : Test Accuracy ≈ 0.8612

```

Conclusion: LSTM/GRU outperform plain SimpleRNN (vanishing gradient on long sequences); Bi-LSTM close to LSTM here, gains typically show more on longer sequences.

Program 8 — Image Generation using GAN

Aim: Build a DCGAN (Generator + Discriminator) to generate MNIST/Fashion-MNIST digit images; evaluate with loss curves, t-SNE/UMAP, precision-recall.

Theory: A GAN pits two networks against each other in a minimax game: the **Generator** maps random noise → fake images, trying to fool the **Discriminator**; the Discriminator tries to correctly classify real vs. fake. Both are trained alternately via backpropagation using their own losses (binary cross-entropy). Over training, the Generator improves until the Discriminator can no longer tell real from fake (its accuracy drifts toward ~50%, i.e., random guessing) — at that point the Generator produces realistic-looking images. **BatchNorm + LeakyReLU** stabilize GAN training; tanh output matches the [-1,1] normalized pixel range.

Dataset loading:

```

import tensorflow as tf
from tensorflow.keras import layers
(x_train, _), (_, _) = tf.keras.datasets.mnist.load_data()
x_train = (x_train.reshape(-1,28,28,1).astype('float32') - 127.5) /
127.5 # scale to [-1,1] for tanh output
BUFFER_SIZE, BATCH_SIZE, NOISE_DIM, EPOCHS = 60000, 256, 100, 30
train_dataset =
tf.data.Dataset.from_tensor_slices(x_train).shuffle(BUFFER_SIZE).batch
(BATCH_SIZE)

```

Generator (Dense→Reshape→Conv2DTranspose upsampling, LeakyReLU, tanh output):

```

def make_generator():
    model = tf.keras.Sequential([
        layers.Dense(7*7*256, use_bias=False,
input_shape=(NOISE_DIM,)),
        layers.BatchNormalization(), layers.LeakyReLU(),
        layers.Reshape((7,7,256)),
        layers.Conv2DTranspose(128,
(5,5),strides=1,padding='same',use_bias=False),
        layers.BatchNormalization(), layers.LeakyReLU(),
        layers.Conv2DTranspose(64,
(5,5),strides=2,padding='same',use_bias=False),
        layers.BatchNormalization(), layers.LeakyReLU(),
        layers.Conv2DTranspose(1,
(5,5),strides=2,padding='same',use_bias=False,activation='tanh')
    ])
    return model

```

Discriminator (Conv2D downsampling + Dropout, linear/logit output):

```

def make_discriminator():
    return tf.keras.Sequential([
        layers.Conv2D(64,(5,5),strides=2,padding='same',
input_shape=[28,28,1]), layers.LeakyReLU(), layers.Dropout(0.3),
        layers.Conv2D(128,(5,5),strides=2,padding='same'),
        layers.LeakyReLU(), layers.Dropout(0.3),
        layers.Flatten(), layers.Dense(1)
    ])

```

Losses + two separate Adam optimizers (key GAN config):

```

cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)
generator_optimizer = tf.keras.optimizers.Adam(1e-4)
discriminator_optimizer = tf.keras.optimizers.Adam(1e-4)

```

```

def generator_loss(fake_out): return
cross_entropy(tf.ones_like(fake_out), fake_out)
def discriminator_loss(real_out, fake_out):
    return cross_entropy(tf.ones_like(real_out), real_out) +
cross_entropy(tf.zeros_like(fake_out), fake_out)

```

Training step (adversarial, two gradient tapes):

```

@tf.function
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE, NOISE_DIM])
    with tf.GradientTape() as gen_tape, tf.GradientTape() as
disc_tape:
        gen_imgs = generator(noise, training=True)
        real_out = discriminator(images, training=True)
        fake_out = discriminator(gen_imgs, training=True)
        g_loss = generator_loss(fake_out)

```

```

        d_loss = discriminator_loss(real_out, fake_out)
        generator_optimizer.apply_gradients(zip(gen_tape.gradient(g_loss,
generator.trainable_variables), generator.trainable_variables))

discriminator_optimizer.apply_gradients(zip(disc_tape.gradient(d_loss,
discriminator.trainable_variables),
discriminator.trainable_variables))

```

Output:

Epoch 1/30: G=0.82 D=0.97 Disc-Acc=0.74
Epoch 30/30: losses stabilize, G≈0.87-0.93, D≈1.2-1.3, Disc-Acc≈0.63-0.69
t-SNE/UMAP: generated-image cluster overlaps real-image cluster (visual similarity)
Precision ≈ 0.84, Recall ≈ 0.86 (real vs fake sample distributions)

Conclusion: Generator progressively fools the Discriminator (its accuracy drifts toward ~50-65%, i.e., ~random), producing recognizable digit-like images by epoch 30.

Program 9 — Text Generation by Fine-Tuning GPT-2 (Shakespeare / custom corpus)

Aim: Fine-tune pretrained GPT-2 on a custom text corpus (e.g., Shakespeare) for creative text generation.

Theory: GPT-2 is a **decoder-only Transformer**, pretrained on huge web text to predict the next token given all previous tokens (causal/autoregressive language modeling, masked self-attention so a token can't see the future). **Fine-tuning** continues this same next-token-prediction objective on a small domain-specific corpus, nudging the model's weights toward that corpus's vocabulary/style without retraining from scratch.

`DataCollatorForLanguageModeling(mlm=False)` prepares batches for causal LM (as opposed to BERT's masked-LM). Sampling parameters (temperature/top-k) control creativity vs. coherence at generation time.

Dataset loading + tokenization:

```

from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM, Trainer,
TrainingArguments, DataCollatorForLanguageModeling

```

```

dataset = load_dataset("text", data_files={"train":
"shakespeare.txt"})
tokenizer = AutoTokenizer.from_pretrained("gpt2")
tokenizer.pad_token = tokenizer.eos_token

```

```

def tokenize_fn(ex):
    r = tokenizer(ex["text"], truncation=True, padding="max_length",
max_length=128)

```

```
r["labels"] = r["input_ids"].copy()
return r
```

```
tokenized = dataset["train"].map(tokenize_fn, batched=True,
remove_columns=["text"])
```

Model + data collator (causal LM, no masking):

```
model = AutoModelForCausalLM.from_pretrained("gpt2")
data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer,
mlm=False)
```

Training configuration (Trainer/TrainingArguments — HF's internal AdamW optimizer):

```
training_args = TrainingArguments(
    output_dir="./gpt2_shakespeare",
    num_train_epochs=1,
    per_device_train_batch_size=2,
    logging_steps=10,
    save_total_limit=1,
    report_to="none"
)
trainer = Trainer(model=model, args=training_args,
train_dataset=tokenized, data_collator=data_collator)
trainer.train()
```

Generation (sampling config):

```
inputs = tokenizer("To be, or not to be", return_tensors="pt")
output = model.generate(**inputs, max_new_tokens=80, do_sample=True,
temperature=0.8, top_k=50, pad_token_id=tokenizer.eos_token_id)
print(tokenizer.decode(output[0], skip_special_tokens=True))
```

Output:

Prompt: "To be, or not to be"

Generated: "To be, or not to be; and to be. Exit ..." (style mimics Shakespearean stage-direction phrasing)

Prompt: "Love is a smoke"

Generated: "Love is a smoke, but a fire. We should have a very long time, ..."

Conclusion: even 1 epoch of fine-tuning shifts GPT-2's generic output toward the target corpus's vocabulary/style; more epochs + full corpus (not just 2000 lines) improve coherence.

Program 10 — LLM Fine-Tuning & Prompt Engineering

Aim: Fine-tune GPT-2 on WikiText-2 and study the effect of different prompting strategies on generation quality.

Theory: Prompt engineering is the practice of crafting the input text (instructions, examples, context) to steer a pretrained/fine-tuned LLM's output without changing its weights. Key generation-time controls: **temperature** scales the logits before softmax (low = confident/deterministic/repetitive, high = diverse/random); **top-k** restricts sampling to the k most-probable next tokens; **top-p (nucleus sampling)** restricts to the smallest set of tokens whose cumulative probability $\geq p$. Combined with fine-tuning (which adapts the model's *knowledge*), prompting adapts how that knowledge is *elicited* at inference time.

Dataset loading:

```
from datasets import load_dataset
dataset = load_dataset("Salesforce/wikitext", "wikitext-2-raw-v1")
train_dataset, eval_dataset = dataset["train"], dataset["validation"]
```

Tokenizer + model (add pad token, resize embeddings):

```
from transformers import GPT2Tokenizer, GPT2LMHeadModel
tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
tokenizer.pad_token = tokenizer.eos_token
model = GPT2LMHeadModel.from_pretrained("gpt2")
model.resize_token_embeddings(len(tokenizer))

def tokenize_fn(ex):
    t = tokenizer(ex["text"], truncation=True, padding="max_length",
max_length=128)
    t["labels"] = t["input_ids"].copy()
    return t

tokenized_train = train_dataset.map(tokenize_fn, batched=True,
remove_columns=["text"])
tokenized_eval = eval_dataset.map(tokenize_fn, batched=True,
remove_columns=["text"])
```

Training config (eval each epoch, mixed precision, AdamW optimizer under the hood):

```
from transformers import Trainer, TrainingArguments,
DataCollatorForLanguageModeling
import torch

training_args = TrainingArguments(
    output_dir="./gpt2_finetuned",
    eval_strategy="epoch", save_strategy="epoch",
    learning_rate=2e-5, per_device_train_batch_size=8,
per_device_eval_batch_size=8,
    num_train_epochs=1, fp16=torch.cuda.is_available(),
weight_decay=0.01,
```

```

        load_best_model_at_end=True, report_to="none"
    )
    trainer = Trainer(model=model, args=training_args,
train_dataset=tokenized_train,
                    eval_dataset=tokenized_eval,
processing_class=tokenizer,

data_collator=DataCollatorForLanguageModeling(tokenizer=tokenizer,
mlm=False))
trainer.train()

```

Prompt-engineering test (temperature/top-k/top-p — the “prompt engineering” part):

```

def generate_text(prompt, temperature=0.8, top_k=50, top_p=0.95):
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    out = model.generate(**inputs, max_new_tokens=100, do_sample=True,
                        temperature=temperature, top_k=top_k,
top_p=top_p,
                        pad_token_id=tokenizer.eos_token_id)
    return tokenizer.decode(out[0], skip_special_tokens=True)

print(generate_text("Once upon a time in a world ruled by AI,"))

```

Output:

Prompt: "Once upon a time in a world ruled by AI,"
Generated: "...human beings have become increasingly aware of their surroundings, and have begun to do more things to ensure that human beings are protected from harm, instead of fearing a sudden invasion by AI. This may be due to their limited ability to reason..."

Conclusion: lower temperature (~0.3-0.5) → safer/repetitive text; higher temperature (~0.8-1.0) → more creative but less coherent; top_k/top_p truncate the sampling pool to balance diversity vs. quality — this is the core “prompt engineering” trade-off to discuss in the answer.

Common exam-answer skeleton (all programs)

1. **Aim** — 1 line from dataset+task given.
2. **Dataset & preprocessing** — load, normalize/scale, reshape, train-test split, one-hot/tokenize as needed.
3. **Model architecture** — Input layer → Hidden/Conv/Recurrent layers (name activation used) → Output layer (softmax for multi-class, sigmoid for binary).
4. **Compilation** — loss function + optimizer (SGD/Adam/AdamW) + metrics.
5. **Training** — epochs, batch size, validation split.
6. **Results** — accuracy/loss numbers, brief comparison table.
7. **Conclusion** — one line linking architecture choice back to the dataset/task.